

Workshop 2

Introduction

This workshop introduces basic formal modelling in Alloy and using the Alloy Analyzer. The aim is to familiarise yourself with Alloy and to apply some of the concepts seen in this week's lectures (sets, relations, etc.) to the basics of writing *formal specifications*. A formal specification is simply a description written in logic about a system and how it should behave.

Your tasks

1. Download Alloy 6.2.0 from the Alloy website: <http://alloytools.org/>

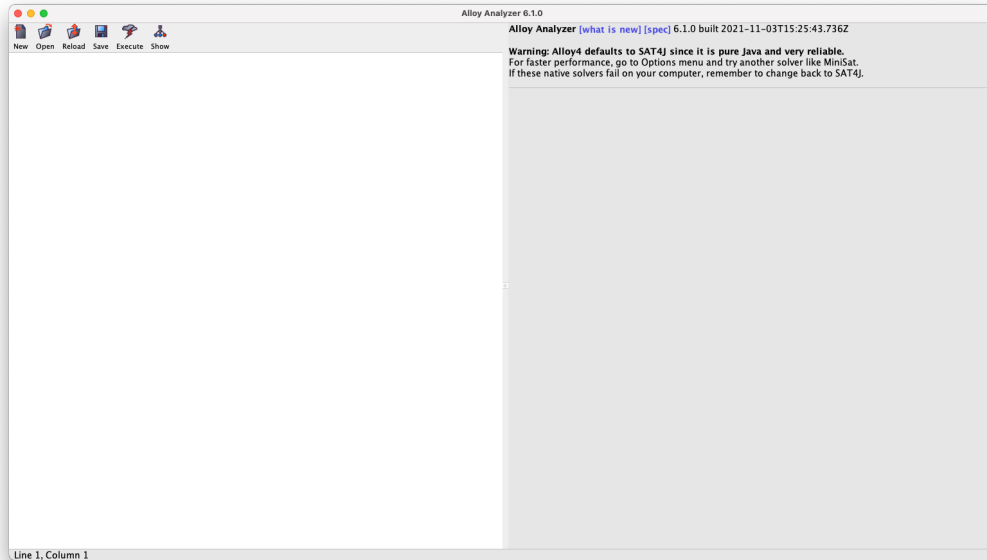
MacOS: If you are running Alloy on MacOS, you can download the appropriate `.zip` file for your hardware (aarch64 if you are on Apple silicon M1, M2, M3 chip etc., or amd64 if you are on Intel hardware). Open or unzip it, which should create an `Alloy.app` application. Drag the `Alloy.app` application to the *Applications* folder to install it. Then run the `Alloy.app` from the *Applications* folder to start Alloy. If you encounter the error message “Alloy.app cannot be opened because the developer cannot be verified” then choose *Cancel / Close* and then open the *System Preferences / Settings* app, go to *Security & Privacy / Privacy & Security*, choose the *General* tab if it exists or just scroll down otherwise and you should see a message similar to “Alloy.app was blocked from use because it is not from an identified developer”. Click *Open Anyway* and then authenticate, or double-click the padlock and enter your password to allow editing, then choose *Open Anyway*, and Alloy should re-open successfully.

Other platforms (but also MacOS too): Otherwise, you can download the download the `org.alloytools.alloy.dist.jar` file. You can run it either by double-clicking the jar file or by running in a terminal/console:

```
java -jar org.alloytools.alloy.dist.jar
```

You may need to install Java first if you don't have it installed already.

In either case, you should see a screen like the following, once Alloy is running:



2. In this workshop we are going to use Alloy to model graphs. Mathematically, recall that a graph is just a pair $\langle V, E \rangle$ where V is a set of *vertices* (aka nodes) and E is a set of *edges*. Each member of E is a pair (v, u) where each of v and u are vertices from V . (v, u) is in E if and only if the graph has an edge from vertex v to vertex u .

We will model this in Alloy using the following *signature* declaration.

```
sig Node {}

one sig Graph {
  edges: Node -> Node
}
```

The first part declares a set `Node` which will be the set of possible nodes in the graph. The second part declares a binary relation `edges` on `Nodes`. Technically it declares a set `Graph` of graphs that has only “one” element (i.e. our model is talking about just a single graph). For any graph g , $g.edges$ is then the edge relation for that graph. Since the set `Graph` has only one element in it, to refer to the set of edges in the graph in our model we can just write `Graph.edges`.

3. Enter the following *predicate* definition, which specifies when a graph g has exactly two edges.

```
pred has_two_edges[g: Graph] {
  # (g.edges) = 2
}
```

We can use the **run** command to view *satisfying instances* of this predicate, i.e. to get Alloy to show us graphs that have exactly two edges. Add the following run command and then choose *Execute* → *Run has_two_edges* from the menu.

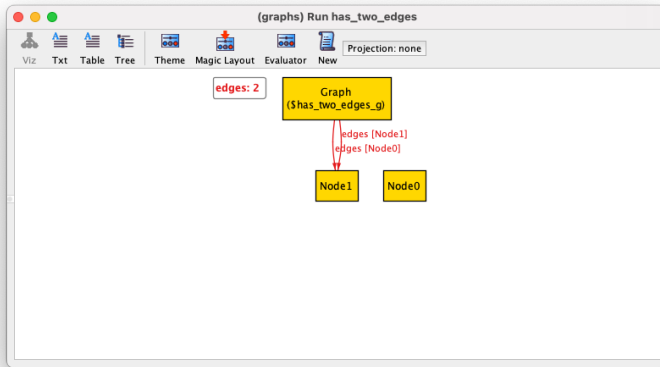
```
run has_two_edges
```

You should see in the right-hand pane output like the following:

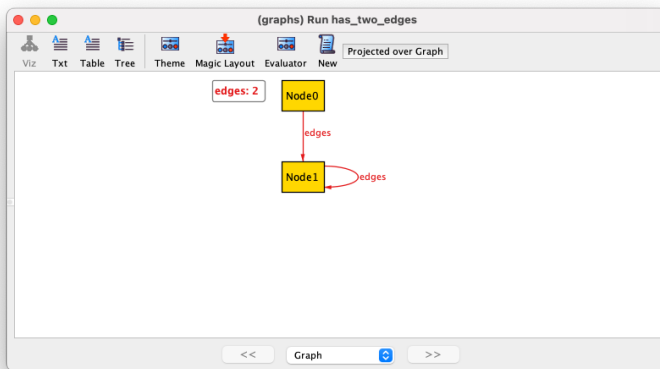
Executing "Run has_two_edges"

```
Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
123 vars. 13 primary vars. 299 clauses. 80ms.
Instance found. Predicate is consistent. 18ms.
```

The word "Instance" is a hyperlink. Click it to view the model that Alloy has found that satisfies this predicate. It should look similar to the following.



We can see here that model shows the single element of the `Graph` set. However since there is only one graph, this extra detail is unnecessarily confusing. Therefore we can essentially filter it out by *projecting* the model over the `Graph` signature. Click the button that says *Projection: none* and choose *Graph*. The output should change to something similar to the following, which is the typical graphical depiction of a graph with two nodes (here called *Node0* and *Node1*, with an edge from *Node0* to *Node1* and a self-loop from *Node1* to itself).



4. Write a predicate `is_undirected` that holds for a graph g when that graph is *undirected*. Recall an undirected graph is one such that whenever it contains an edge (v, u) it also contains the symmetric edge (u, v) .

Use the **run** command to view instances of your predicate.

5. The following two functions are shorthand respectively that given a graph g and node v , return the set of nodes with edges *to* v , and those with edges *from* v respectively.

```
fun edges_to[g: Graph, v: Node]: set Node {
  g.edges.v
```

```

}

fun edges_from[g: Graph, v: Node]: set Node {
  g.edges[v]
}

```

Using these functions, write a predicate `is_undirected2` that holds for a graph g precisely when g is undirected.

6. The following assertion asserts that the two definitions `is_undirected` and `is_undirected2` are logically equivalent.

```

assert is_undirected_equiv {
  all g: Graph | (is_undirected[g] => is_undirected2[g]) and
                  (is_undirected2[g] => is_undirected[g])
}

```

We can use Alloy's **check** command to check whether this assertion holds, i.e. whether the two definitions really are logically equivalent.

```
check is_undirected_equiv for 20
```

Here we are instructing Alloy to check it for models of size up to 20 members of each signature, i.e. graphs that have at most 20 nodes. Enter this command and then execute the check by choosing *Execute* → *Check is_undirected_equiv for 20* from the menu. Alloy should not find any counter-example (i.e. any model in which this assertion is false), and should produce output like the following:

```

Executing "Check is_undirected_equiv for 20"
  Solver=sat4j Bitwidth=4 MaxSeq=7 SkolemDepth=1 Symmetry=20 Mode=batch
  5396 vars. 421 primary vars. 11595 clauses. 449ms.
  No counterexample found. Assertion may be valid. 125ms.

```

Notice the word “may” in the final line. This means that the assertion might be true always, but Alloy is not 100% certain because it has only checked for models up to size 20. While *this* particular assertion holds in all models regardless of their size, Alloy cannot be certain of this because it only does bounded checking. This is important to remember when using Alloy.

7. Alloy **fact** declarations declare axioms, i.e. assumptions that Alloy will assume are always true. When generating instances of a model or searching for counter-examples to an assertion, Alloy will only consider ones that satisfy the assumptions stated in **fact** declarations.

Add the following fact to your model:

```

fact at_least_one_edge {
  all g : Graph | some g.edges
}

```

What does this fact say?

Modify this fact to instead say that every graph has exactly three (3) edges. Then re-run the `has_two_edges` predicate. What happens? Why?

8. *If you have time:* What do the following predicates mean? Enter each and use Alloy's **run** command to help work out what they are saying logically. Write down your answer as comments in your Alloy file.

```
pred what_does_this_mean1[g: Graph] {  
  all u, v: Node | (u -> v) in g.edges and (v -> u) in g.edges  
}  
  
pred what_does_this_mean2[g: Graph] {  
  all u, v: Node | (u -> v) in g.edges or (v -> u) in g.edges  
}
```